



DATA INTEGRITY AND PROTECTION

Traditional Disk Failure Model: Fail Stop Model

- Disk either **works** or **completely fails**
- Failure is **easy to detect**
- Used in early **RAID systems**
- Simplifies reliability design

Modern Disk Failures

- Disks may **partially fail**: most of the disk works, some blocks don't
- **Latent Sector Errors (LSEs)**: Block inaccessible, Disk sector(s) become **unreadable or damaged**
 - **Causes**
 - Head crash (physical damage), Cosmic rays flipping bits, Media degradation
 - **Detection:**
 - Disk **ECC (Error Correcting Codes)** may detect and fix
 - If unrecoverable, disk returns a **read error**
- **Block Corruption** – Wrong data returned (silent error)
 - **Causes:**
 - **Firmware bugs** (writes wrong location)
 - **Bus errors** (corrupted data during transfer)
 - **Problem:**
 - **Silent faults** – no error reported to the OS
 - Extremely **dangerous** for data integrity

Failure Types, Frequency & Findings

Failure Type	Disk Response	Detectable?	Example Cause
LSE (non-silent)	Returns error	✓ Yes	Head crash
Corruption (silent)	Returns wrong data	✗ No	Firmware bug

Failure Type	Cheap Drives	Costly Drives
LSEs	9.40%	1.40%
Corruption	0.50%	0.05%

- **LSEs and corruptions are common even in costly disks**, often showing spatial and temporal locality and increasing with disk age or size.
- **LSEs** tend to cluster and recur; **disk scrubbing** is effective at finding them.
- **Corruption rates vary by model**, are mostly few in number, and not fully independent across disks.
- Reliable storage systems must include **error detection, redundancy, and recovery** to handle these partial failures.

Handling Latent Sector Errors (LSEs)

- LSEs are easy to detect, so systems can recover using redundancy mechanisms:
 - Mirrored RAID: Read from the alternate copy.
 - RAID-4 / RAID-5: Reconstruct missing data using parity.
- Problem during rebuilds:
 - When one disk fails, RAID reads all other disks to rebuild it.
 - If an LSE occurs on any remaining disk, reconstruction may fail, leading to data loss.
- Solution – Extra Redundancy:
 - NetApp's RAID-DP adds a second parity disk (like RAID-6).
 - Can recover from a full-disk failure plus an LSE.
 - Slightly higher space and write overhead, but significantly better fault tolerance.
- Key Idea:
 - LSEs are recoverable if redundancy exists.
 - Double parity makes RAID robust against combined disk and sector failures.

Detecting Corruption: The Checksum

- The Problem

- Silent failures: Disks can return bad data without reporting errors.
- Goal: Detect corruption to preserve data integrity.
- Recovery: Once detected, corrupted data is replaced using redundancy.

- Checksums: How They Work

- Compute a small summary of a data block (e.g., 4–8 bytes for a 4KB block).
- Store checksum with the block.
- On read, recompute checksum and compare → detect corruption.

Example

- Data block (16 bytes):
 - 365e c4cd ba14 8a92 ecef 2c3a 40be f666
- XOR all bytes → checksum = 0x201b9403
- Limitation: **cannot detect multi-bit errors in same positions**

```
0011 0110 0101 1110    1100 0100 1100 1101
1011 1010 0001 0100    1000 1010 1001 0010
1110 1100 1110 1111    0010 1100 0011 1010
0100 0000 1011 1110    1111 0110 0110 0110

0010 0000 0001 1011    1001 0100 0000 0011
```

Addition Checksum

- Addition checksum = $d_1 + d_2 + d_3 + \dots + d_n$
- On read, you recompute the sum and compare with the stored checksum.
- If the sums match, you assume the data is correct.
- Issues: Cannot detect data shift
 - Suppose the data bytes are: $[1, 2, 3]$, Checksum = $1 + 2 + 3 = 6$
 - Now, the data is **shifted** e.g., $[2, 1, 3]$, Checksum = $2 + 1 + 3 = 6$

Fletcher Checksum

- Computes **two sums**: $s1$ and $s2$
- Algorithm for block $D = d1 \dots dn$:
 - $s1 = (s1 + d_i) \bmod 255$
 - $s2 = (s2 + s1) \bmod 255$
- Detects all single-bit, double-bit errors, and many burst errors.
- Almost as strong as CRC, but faster to compute.

Example: Data Bytes = [1, 2, 3]

Step	d _i	s1 (s1_prev + d _i) mod 255	s2 (s2_prev + s1) mod 255
0	-	0	0
1	1	(0 + 1) = 1	(0 + 1) = 1
2	2	(1 + 2) = 3	(1 + 3) = 4
3	3	(3 + 3) = 6	(4 + 6) = 10

- If a byte is **changed**, s1 changes → checksum changes.
- If bytes are **swapped**, s2 changes even if s1 stays same.
- More robust than simple addition checksum.

Example: Data Bytes = [2, 1, 3]

Step	Byte (d _i)	s1 (s1_prev + d _i) mod 255	s2 (s2_prev + s1) mod 255
0	-	0	0
1	2	0 + 2 = 2	0 + 2 = 2
2	1	2 + 1 = 3	2 + 3 = 5
3	3	3 + 3 = 6	5 + 6 = 11

- [1, 2, 3] gave (6, 10) and [2, 1, 3] gives (6, 11).
- This shows that **Fletcher checksum can detect changes in order of bytes**, not just single-bit errors.

Cyclic Redundancy Error

- Detect data corruption in storage or transmission.
- Treat the data block as a large binary number.
- Choose a generator polynomial (agreed-upon value, e.g., 1011 for 4-bit CRC).
- Append zeros equal to the degree of the generator.
- Perform binary division of data by generator.
- The remainder is the CRC checksum.
- Store or transmit data + CRC.
- At the receiver, divide received data + CRC by the same generator.
 - If remainder = 0 → no errors detected
 - If remainder $\neq$ 0 → error detected

Example: 11010110

- **Data (8-bit):** 11010110
- **Generator (4-bit):** 1011 (degree = 3)
- **Append zeros** equal to generator degree → 11010110000
- **Divide** the data by the generator using binary division.
- **Remainder** after division = 111 → This is the CRC.
- **Transmit:** Original data + CRC → 11010110 111
- **Receiver Check:**
 - Divide 11010110111 (data + CRC) by 1011
 - Remainder = 0 → No error detected

Checksum layout in Storage systems

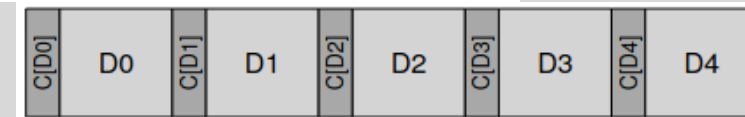
- Store checksums to detect data corruption.

- Compute checksum $C(D)$ for each data block D .

- Layout without checksum:



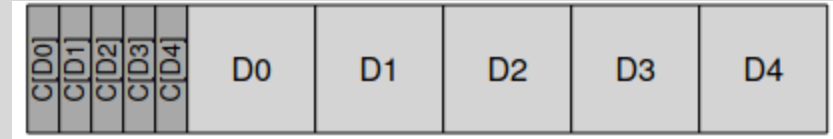
- Layout with per-block checksum:



- Checksums are small (e.g., 8 bytes), but disk writes happen in sector-sized chunks (512 bytes)
- **520-byte sector disks:** Extra 8 bytes per sector for checksum.

-Continue

- In standard disks that only support 512-byte sectors; no extra space for per-sector checksums.
- Pack multiple checksums together in a sector, followed by their corresponding data blocks:



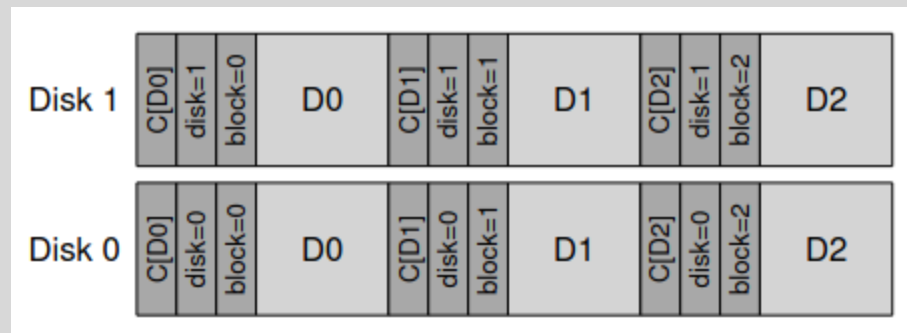
- Works on all standard disks.
- **Cons:** Less efficient: updating a block (e.g., D1) requires:
1 read + 2 writes (read checksum sector, update checksum, write checksum + data).

Using Checksums to detect error

- Read data block D and stored checksum $C_s(D)$.
- Compute checksum $C_c(D)$ for the retrieved data.
- Compare:
 - $C_s(D) == C_c(D) \rightarrow$ Data OK
 - $C_s(D) \neq C_c(D) \rightarrow$ Data corrupted
- Recovery:
 - Use redundant copy if available
 - Otherwise, return an error

Misdirected Writes

- Data written to wrong location
 - Single-disk: Block D_x written to address $y \rightarrow$ corrupts D_y
 - Multi-disk: Block $D_{i,x}$ written to wrong disk j
- **Solution:** Add physical identifiers to checksum
 - Store **disk number** and **block/sector number** with checksum
 - Example: $C(D) + \text{disk ID} + \text{sector offset}$
- **Detection:**
 - If stored location $\neq$ expected location $\rightarrow$ misdirected write detected
- Extra redundancy helps error detection and recovery



Lost Writes

- Device reports write completed, but data not actually stored
 - Old data remains on disk → checksum and physical ID appear correct
- **Challenge:** Basic checksums **cannot detect lost writes**
- **Solutions:**
 - **Write-verify / Read-after-write:** Immediately read block after write to confirm
 - Accurate but doubles I/O → slower writes
 - **Redundant checksums elsewhere:**
 - Example: ZFS stores checksums in **inodes and indirect blocks**
 - Lost write detected if inode checksum $\neq$ block checksum
- **Preventive Measure: Disk scrubbing**
 - Periodically read all blocks and verify checksums
 - Helps catch silent corruption and bit rot

Summary

- Checksums are essential for **detecting data corruption** in storage systems.
- Different checksum methods (simple sum, Fletcher, CRC) protect against different types of errors.
- Modern storage introduces **new failure modes** like misdirected writes and lost writes.
- Solutions include **write-verify, redundant checksums, and disk scrubbing**.
- As storage technology evolves, **new data protection methods may be required**.
- Checksums are powerful, but no method is perfect, ongoing innovation is needed for reliable data protection.



Thank You